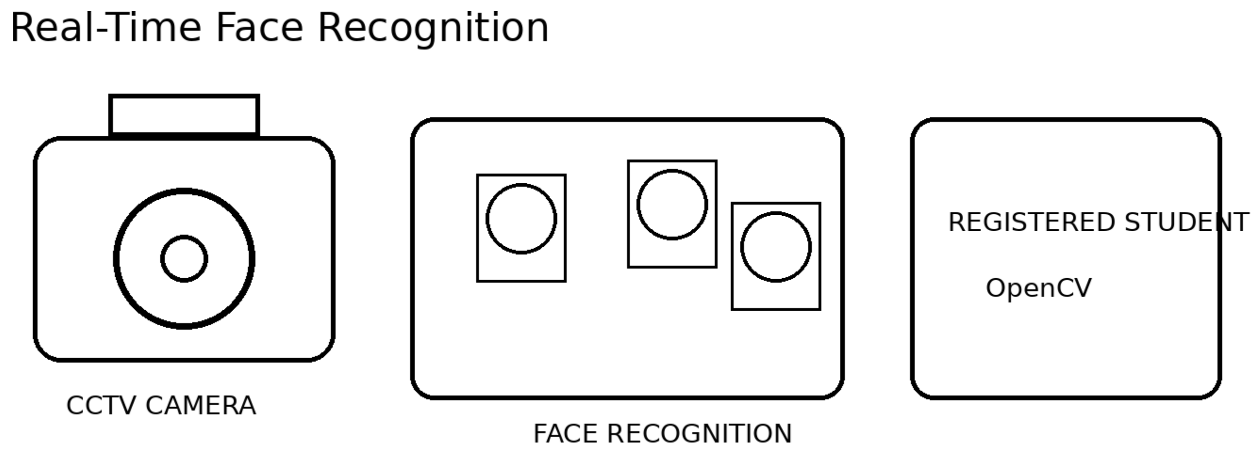
CO5_ASSESSMENT 1

CONSTRAINT -BASED PROBLEM SOLVING

1. Real-Time Face Recognition for Campus Security



System Design

- Image acquisition: OpenCV VideoCapture reads the CCTV stream. Frames are resized to a moderate resolution to reduce CPU usage.
- Preprocessing: Convert to grayscale where suitable and use histogram equalization or CLAHE to improve contrast. Gaussian/median filtering reduces camera noise.
- Face detection: Haar Cascade or an OpenCV DNN face detector identifies face regions. Very small or low-confidence detections are rejected.
- Feature extraction: Align the face using facial landmarks when available, resize it consistently and

generate features using LBPH or a face-embedding model.

- Recognition: Compare the live feature with registered features using distance or similarity. If similarity is insufficient, display Unknown instead of forcing an identity.
- False-recognition control: Confirm an identity over several consecutive frames, use quality checks and conservative thresholds, and maintain an Unknown category.
- Real-time optimization: Detect every few frames, track faces between detections, process only face crops, resize frames and avoid unnecessary disk operations.
- Security: Store only required templates and event records, protect the database and provide manual verification for uncertain cases.

Real-Time Optimization

Real-time performance can be improved by detecting faces only once every few frames and tracking them between detections. The entire image does not need to be processed for recognition every time. Only the detected face region is passed to the recognition algorithm.

Other optimization techniques include:

- Resize input frames.
- Use lightweight face detectors.
- Process every 2nd or 3rd frame.
- Track detected faces between detection frames.
- Avoid unnecessary image saving.
- Use separate threads for video capture and processing.
- Store only required facial templates.

Accuracy and Real-Time Justification: Lighting normalization, consistent face alignment and discriminative features improve recognition accuracy. Periodic detection, tracking and lightweight processing reduce computational load, allowing the system to run with low delay on a low-cost computer.

Final Identification

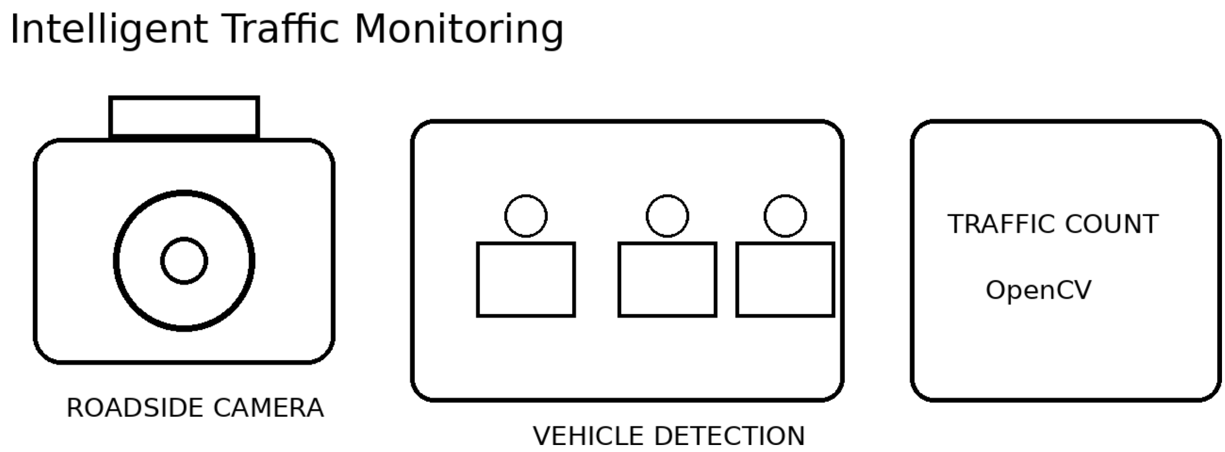
The final output can display the person's name, confidence score, bounding box, date and time. For security purposes, the system can maintain a log of recognized and unknown persons.

Accuracy and Performance

Accuracy is improved through preprocessing, face alignment, suitable feature extraction, confidence thresholds and multi-frame confirmation. Real-time performance is achieved through image resizing, periodic detection, tracking and lightweight OpenCV operations. Therefore, the system can provide a practical solution for campus entrance monitoring even on a low-cost computer.

2. Intelligent Traffic Violation Detection System

A roadside OpenCV system can monitor a defined road region, detect vehicles, maintain their identities across frames, detect line crossings and analyze movement. A lightweight object detector can be combined with OpenCV tracking and image-processing operations.



System Design

- Video acquisition: Read live roadside video using VideoCapture and define a Region of Interest containing only the road.
- Preprocessing: Resize frames and apply Gaussian blur to suppress sensor noise before motion analysis.
- Vehicle detection: Use MOG2/KNN background subtraction for fixed cameras, followed by morphology. For reliable vehicle classes, integrate a lightweight object detector.
- Tracking: Assign each vehicle an ID using centroid matching, bounding-box overlap or a lightweight tracker such as CSRT/KCF.
- Line crossing: Define a virtual line. Compare the previous and current centroid; when a track crosses the line, increase the count only once.

- Abnormal movement: Analyze direction, trajectory and approximate speed. Wrong-way movement or entry into a restricted region can generate an alert.
- Error reduction: Use ROI restriction, minimum object area, morphological filtering, shadow suppression and temporal persistence to reduce false detections.
- Multiple vehicles: Maintain separate track IDs and tolerate short occlusions so that vehicles are not counted repeatedly.
- Visualization: Draw boxes, IDs, counting line, total count and alert messages directly on the live frame.

Handling Multiple Vehicles

When many vehicles are present, every vehicle must receive a separate tracking ID. Centroid distance and bounding-box overlap can be used to associate detections with existing tracks.
Short-term occlusion handling is important. If two vehicles overlap for a few frames, their tracks should not immediately be deleted.

Visualization

The output frame can display:

- Vehicle bounding boxes.
- Vehicle ID.
- Counting line.
- Vehicle count.
- Direction.
- Violation alert.

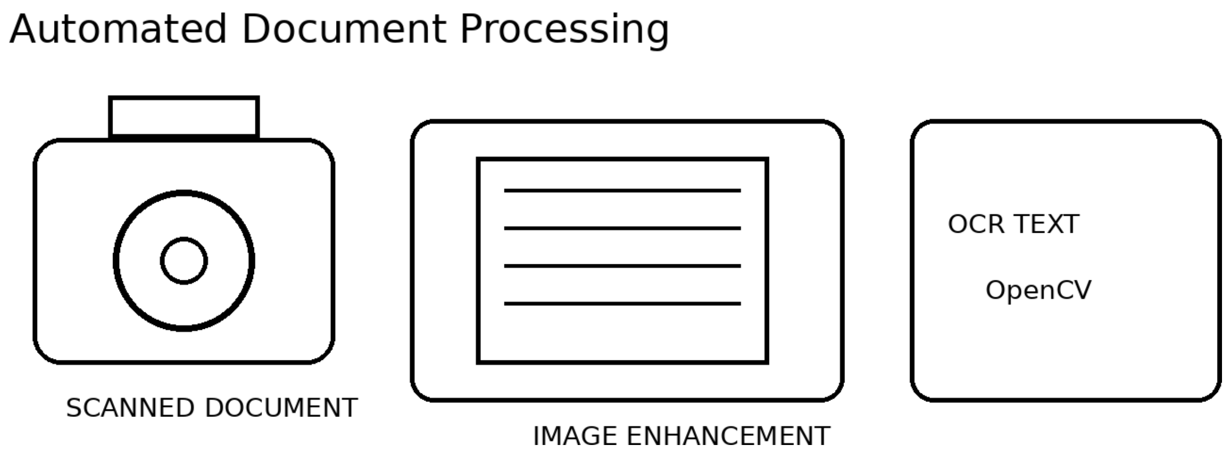
Functions such as cv2.rectangle(), cv2.line() and cv2.putText() can be used.

Accuracy and Performance
Counting vehicles based on tracked trajectories is more reliable than simply detecting vehicles in every frame. ROI restriction, filtering, shadow suppression and temporal confirmation reduce errors. Processing only important road regions and using lightweight tracking allows the system to operate in real time.

Accuracy and Real-Time Justification: Counting is based on tracked trajectories rather than individual frames, which prevents repeated counting. ROI restriction, filtering and temporal confirmation reduce shadows and
dynamic-background errors, while tracking supports accurate multi-vehicle monitoring.

3. Automated Document Processing System

The document pipeline prepares poor-quality scanned pages for OCR. OpenCV performs grayscale conversion, denoising, illumination correction, thresholding, morphology, geometric correction and text-region segmentation before the cleaned image is passed to OCR.



System Design

- Image acquisition: Read scanned pages with OpenCV and preserve enough resolution for small characters.
- Grayscale conversion: Remove unnecessary color information and simplify subsequent image processing.
- Denoising: Apply median filtering for salt-and-pepper noise or Gaussian/bilateral filtering for other noise, while preserving character strokes.
- Illumination correction: Reduce shadows and uneven lighting using background normalization or adaptive thresholding.

- Thresholding: Use Otsu for uniform pages and adaptive thresholding for uneven illumination to separate text from background.
- Morphological operations: Opening removes isolated specks; closing reconnects broken strokes and fills small gaps.
- Geometric correction: Estimate page skew and rotate the image. Perspective transformation can convert a photographed page into a rectangular document.
- Segmentation: Use contours, connected components or projection profiles to isolate text blocks, lines and words.
- Batch processing: Process documents in batches, log failures, reuse processing parameters and use parallel processing when hardware allows.

Text Segmentation

The document can be divided into text regions using contours or connected components.

Large regions can represent:

- Paragraphs.
- Headings.
- Tables.
- Images.
- Text lines.

Accuracy and Real-Time Justification: Each stage removes a specific OCR difficulty: noise, poor contrast, uneven background, broken strokes, skew or unwanted regions. The final high-contrast, geometrically corrected text image is easier for OCR to interpret accurately and efficiently.

Batch Processing

Since thousands of documents may need processing, the system should support batch processing.

Files can be processed one after another automatically. Failed files can be recorded in a log. Multiple documents can also be processed in parallel when sufficient CPU resources are available.

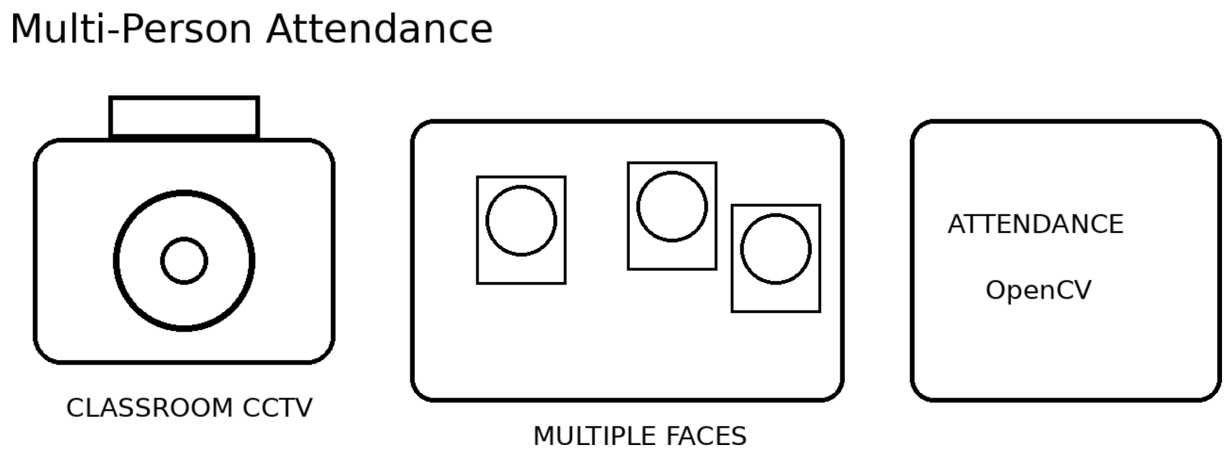
Accuracy Improvement

Each stage solves a specific problem. Grayscale simplifies processing, filtering removes noise, thresholding separates text, morphology repairs character structures, and geometric correction fixes skew and perspective.

As a result, the OCR engine receives a clean, high-contrast and properly aligned document, increasing recognition accuracy.

4. Real-Time Multi-Person Tracking and Attendance System

A classroom attendance system detects multiple faces, recognizes them and maintains a track identity over time. Recognition is performed only when needed, while tracking handles the frames between detections.



System Design

- Video acquisition: Capture classroom CCTV frames and resize them for real-time processing.
- Face detection: Detect all visible faces using an OpenCV DNN detector or Haar cascade and reject poor-quality or tiny faces.
- Feature extraction: Align and resize each face and create a feature representation using LBPH or face embeddings.
- Recognition: Compare each feature against registered student templates. Low-confidence matches remain Unknown.
- Tracking: Assign a persistent track ID using centroid association or a lightweight tracker such as CSRT/KCF.

- Temporary disappearance: Keep a track alive for a short timeout when a student turns away or is briefly occluded.
- Duplicate prevention: Maintain an attendance set for the current class session. Once identity is confirmed, mark attendance only if that ID is not already recorded.
- False-match control: Require agreement over several frames and use face-size and quality thresholds.
- Real-time optimization: Detect periodically, track between detections and recognize only cropped face regions.

Tracking

Tracking allows the system to follow each person between frames.

Every detected person receives a track ID.

For example:

Track 01 → Student A

Track 02 → Student B

Track 03 → Student C

The tracker stores position and last-seen time.

Temporary Face Disappearance

A student may turn their head, move behind another student or temporarily leave the camera view.

The system should not immediately delete the identity.

A short timeout can be used. If the person reappears near the expected location, the same track can be restored.

10. Real-Time Optimization

Real-time performance can be improved by:

- Detecting faces every few frames.
- Tracking between detections.
- Processing only face crops.
- Resizing frames.
- Using lightweight detectors.
- Avoiding unnecessary image storage.
- Using multi-threaded video capture.

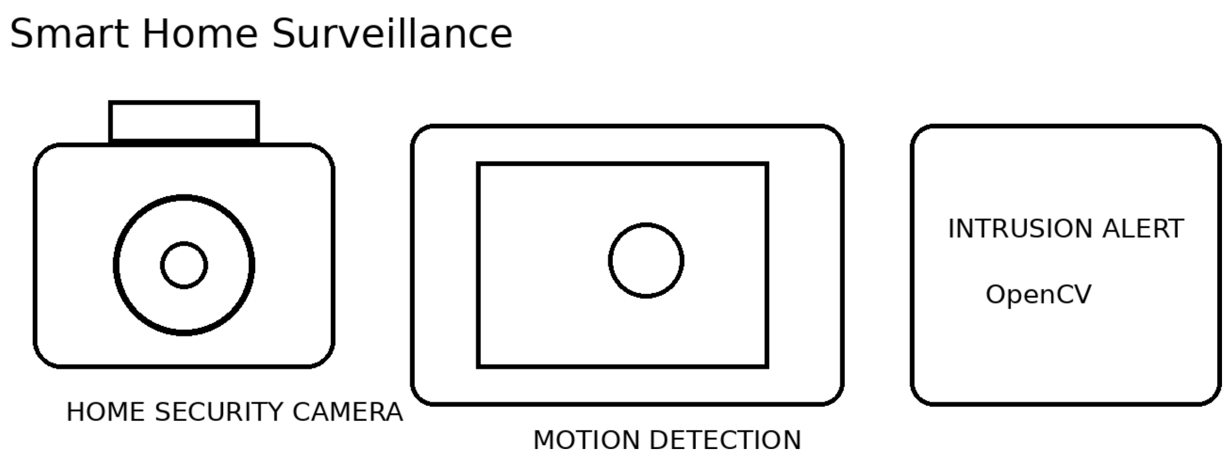
Accuracy and Performance

The combination of detection, feature extraction, recognition and tracking provides stable identity management. Multi-frame confirmation prevents false attendance, while the attendance set prevents duplicates.

Temporary track retention handles slight head movement and short disappearance. Periodic detection and tracking reduce computational requirements and allow continuous classroom operation.

5. Smart Home Surveillance and Intrusion Detection

A fixed-camera home surveillance system can detect significant motion using background subtraction, clean the foreground mask, identify moving objects through contours, track them and generate an alert only when a persistent object enters a protected entrance region.



System Design

- Video acquisition: Read the continuous camera stream and define the entrance ROI plus optional exclusion regions.
- Background subtraction: MOG2 or KNN produces a foreground mask representing changes from the learned background.
- Filtering: Apply Gaussian or median blur to suppress camera noise before motion analysis.
- Morphological processing: Opening removes isolated pixels; closing connects fragmented foreground regions.
- Shadow reduction: Use MOG2 shadow information and brightness/chromaticity checks where needed to prevent shadows becoming false objects.
- Contour detection: Find contours and calculate bounding boxes, area and centroids. Ignore very small objects.

- Motion analysis: Track centroid displacement and require persistence across several frames before declaring significant motion.
- Tracking: Assign temporary IDs to moving objects and monitor their position relative to the entrance ROI.
- Alert: Trigger an intrusion alert only when a persistent tracked object enters the protected region and satisfies the defined time/area conditions.

Real-Time Performance

The system mainly uses lightweight OpenCV operations, so it can run on a low-cost computer.

Performance can be improved by:

- Reducing frame resolution.
- Processing only the entrance ROI.
- Using efficient background subtraction.
- Avoiding unnecessary image storage.
- Tracking objects instead of repeatedly detecting them.

Accuracy and Real-Time Justification: The system is computationally light because most operations are classical OpenCV techniques. Spatial ROI rules, minimum-area filtering, shadow suppression and temporal persistence reduce false alarms from noise, illumination changes, trees and other dynamic background objects.